# Problem Set 9

Foundations of Machine Learning

- **Submission**: Turn in as a `PDF` and a **separate source code file** on `http://mycourses.unh.edu`. Scanned handwritten notes are encouraged.
- **Questions**: See the class website for a link to office hours.

**Relevant reading**:

(a) UDL (Prince, *Understanding Deep Learning*): Chapters 2 and 3, Section 5.5 (Softmax and multiclass classification), and Sections 7.1–7.4 (Backpropagation), available at `https://udlbook.github.io/udlbook/`

(b) You can also check out notebooks that illustrate the concepts: `https://udlbook.github.io/udlbook/`

## Problem 1

Recall that the *softmax* function $\text{softmax}\colon \mathbb{R}^K \to \mathbb{R}^K$ maps a vector $\boldsymbol{z} = (z_1, \ldots, z_K)$ to a vector of probabilities with the components

$$(\text{softmax}[\boldsymbol{z}])_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \qquad k = 1, \ldots, K.$$

Show that softmax is *translation invariant*: for any $c \in \mathbb{R}$,

$$\text{softmax}[\boldsymbol{z} + c\mathbf{1}] = \text{softmax}[\boldsymbol{z}],$$

where $\mathbf{1} = (1, \ldots, 1)$ is the vector of all ones.

## Problem 2

Consider a classification problem with two classes $y \in \{0, 1\}$ and features $\boldsymbol{a} \in \mathbb{R}^n$. Assume that $a_1 = 1$ represents the intercept. A *linear softmax model* predicts the probability of a data point $\boldsymbol{a}$ being in class $k$ as

$$\hat{y}^{\text{s}}_k(\boldsymbol{w}) = (\text{softmax}[(z_0, z_1)])_k, \qquad z_k = \boldsymbol{a}^\top \boldsymbol{w}_k, \qquad k \in \{0, 1\},$$

where $\boldsymbol{w} = (\boldsymbol{w}_0, \boldsymbol{w}_1)$ are the learned coefficients. Note that, unlike in Problem 1, the components of softmax are indexed from 0 here, so $K = 2$ and $(\text{softmax}[(z_0, z_1)])_k = \exp(z_k)/(\exp(z_0) + \exp(z_1))$. On the same classification problem, *logistic regression* predicts the probability of each class as

$$\hat{y}^{\text{l}}_1(\boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{a}^\top \boldsymbol{x}}}, \qquad \hat{y}^{\text{l}}_0(\boldsymbol{x}) = 1 - \hat{y}^{\text{l}}_1(\boldsymbol{x}).$$

(a) Use Problem 1 to explain why there are many values of $\boldsymbol{w}$ that lead to identical predictions of the linear softmax model.

(b) Show that for any weights $(\boldsymbol{w}_k)_{k\in\{0,1\}}$ in the softmax linear model, there exist weights $\boldsymbol{x}$ in logistic regression such that $\hat{y}^{\text{s}}_k(\boldsymbol{w}) = \hat{y}^{\text{l}}_k(\boldsymbol{x})$ for all $k \in \{0, 1\}$ and all $\boldsymbol{a}$.

(c) Show that for any weights $\boldsymbol{x}$ in logistic regression, there exist weights $(\boldsymbol{w}_k)_{k\in\{0,1\}}$ in the softmax linear model such that $\hat{y}^{\text{s}}_k(\boldsymbol{w}) = \hat{y}^{\text{l}}_k(\boldsymbol{x})$ for all $k \in \{0, 1\}$ and all $\boldsymbol{a}$.

(d) Show that the cross-entropy loss of the softmax model equals the logistic regression loss for the same predictions: $\hat{\boldsymbol{y}}^{\text{s}}(\boldsymbol{w}) = \hat{\boldsymbol{y}}^{\text{l}}(\boldsymbol{x})$. It is sufficient to argue the equivalence for a single data point with features $\boldsymbol{a} \in \mathbb{R}^n$ and a label $y_{\text{true}}$. Recall that for a single data point, the cross-entropy loss is defined as $H(y, p) = -y \log p - (1 - y) \log(1 - p)$.

(e) Argue that both models make the same predictions after fitting.

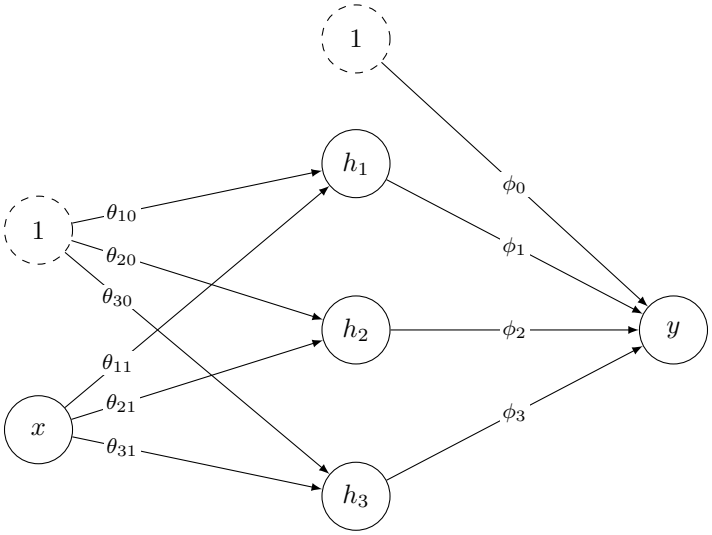



Figure 1: Network used in Problem 4.

# Problem 3

Problems 3.1, 3.2, 3.3, 3.4 in UDL (see above for the link to the book).

# Problem 4

Note that UDL writes function arguments in square brackets, such as $f[x, \boldsymbol{\phi}]$ and $a[z]$, to avoid confusing function application with multiplication. It is not a standard convention and the assignment uses parentheses here.

Consider the shallow neural network from UDL Chapter 3 with a scalar input $x$, three hidden units $h_1, h_2, h_3$, and a scalar output $y$:

$$h_1 = a(\theta_{10} + \theta_{11}x), \qquad h_2 = a(\theta_{20} + \theta_{21}x), \qquad h_3 = a(\theta_{30} + \theta_{31}x),$$

$$y = f(x, \boldsymbol{\phi}) = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3,$$

where $a(z) = \mathrm{ReLU}(z) = \max\{0, z\}$ is the activation function and $\boldsymbol{\phi} = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$ are the 10 parameters. The network is depicted in Figure 1.

For a single training example $(x, y_{\text{true}})$, the least-squares loss is

$$\ell(\boldsymbol{\phi}) = \left(f(x, \boldsymbol{\phi}) - y_{\text{true}}\right)^2.$$

Use the parameter values

$$\theta_{10} = 0.5, \quad \theta_{11} = 1, \quad \theta_{20} = -0.5, \quad \theta_{21} = -1, \quad \theta_{30} = -1, \quad \theta_{31} = 2,$$
$$\phi_0 = 0.2, \quad \phi_1 = 1, \quad \phi_2 = -1, \quad \phi_3 = 0.5,$$

and the training example $x = 1$, $y_{\text{true}} = 1$.

(a) Compute the pre-activations $\theta_{d0} + \theta_{d1}x$, the hidden units $h_1, h_2, h_3$, the output $f(x, \boldsymbol{\phi})$, and the loss $\ell(\boldsymbol{\phi})$.

(b) Use the chain rule to derive general expressions for the derivatives $\partial\ell/\partial\phi_d$ for $d = 0, \ldots, 3$ and $\partial\ell/\partial\theta_{d0}$, $\partial\ell/\partial\theta_{d1}$ for $d = 1, 2, 3$. Recall that $a'(z) = 1$ when $z > 0$ and $a'(z) = 0$ when $z < 0$.

(c) Evaluate all 10 derivatives from part (b) at the parameter values above. Which derivatives are zero and why?

(d) **[code]** Check your answer to part (c) numerically using automatic differentiation (as in previous problem sets).

## Time spent

Report approximately how many hours you spent completing this problem set (including the reading).